

Sistemas embebidos

Instrucciones de lectura y escritura

μ C ATmega328P

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2025-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

Microcontrolador ATmega328P

Características principales:

- Arquitectura RISC de 8 bits
- 32 registros de propósito general
- 32KB de memoria Flash
- 2KB de SRAM
- 1KB de EEPROM
- 23 pines de E/S
- Velocidad hasta 20 MHz



Nota

El ATmega328P es el **corazón** de **Arduino UNO**

Espacios de Memoria

Arquitectura Harvard Modificada

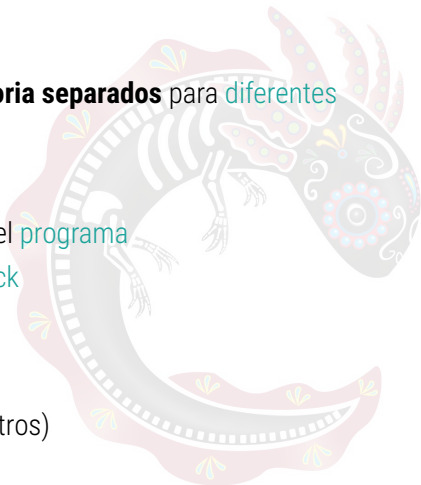
El ATmega328P tiene **espacios de memoria separados** para diferentes propósitos

Tres espacios de memoria principales:

- **Memoria Flash (32KB):** Almacena el programa
- **SRAM (2KB):** Datos variables y stack
- **EEPROM (1KB):** Datos no volátiles

Espacios de direccionamiento:

- **Registros:** 0x00 - 0x1F (32 registros)
- **E/S:** 0x20 - 0x5F (64 registros)
- **E/S Extendida:** 0x60 - 0xFF
- **SRAM:** 0x0100 - 0x08FF



Registros de Trabajo

32 Registros de Propósito General

Los registros R0 a R31 son la base del **procesamiento de datos**

Registros especiales:

- **R26, R27 (X):** Apuntador X para direccionamiento indirecto
- **R28, R29 (Y):** Apuntador Y para direccionamiento indirecto
- **R30, R31 (Z):** Apuntador Z para direccionamiento indirecto

Ejemplo de uso:

```
1 ; Cargar valor inmediato en registro
2 LDI r16, 0xFF          ; Cargar 255 en R16
3 LDI r17, 0x00          ; Cargar 0 en R17
4
5 ; Mover entre registros
6 MOV r18, r16           ; Copiar R16 a R18
```

Instrucción LDI - Load Immediate

LDI Rd, k

Carga un **valor inmediato (constante)** en un **registro**

Características:

- Solo funciona con registros **R16-R31**
- El valor **k** es de **8 bits (0 a 255)**
- Tiempo de ejecución: **1 ciclo de reloj**

Ejemplos:

```
1 LDI r16, 0xFF          ; Cargar 255 decimal
2 LDI r17, 0b11110000    ; Cargar binario
3 LDI r18, 0xAB          ; Cargar hexadecimal
4 LDI r19, 100           ; Cargar decimal
5 LDI r20, 'A'           ; Cargar caracter ASCII
```

Instrucción LDS - Load Direct from SRAM

LDS Rd, k

Carga un **byte** desde la **SRAM** a un **registro**

Características:

- Acceso directo a **cualquier posición** de **SRAM**
- Dirección *k* puede ser de 0x0000 a 0xFFFF
- Tiempo de ejecución: 2 ciclos de reloj

Ejemplos:

```

1 LDS r16, 0x0100      ; Leer desde direccion 0x0100
2 LDS r17, 0x0200      ; Leer desde direccion 0x0200
3 LDS r18, variable     ; Leer variable definida
4
5 ; Definicion de variable en SRAM
6 .dseg
7 variable: .byte 1      ; Reservar 1 byte
  
```

Instrucción LD - Load Indirect

LD Rd, X/Y/Z

Carga un **byte** usando **direccionamiento indirecto** con **apuntadores**

Variantes disponibles:

- LD Rd, X: Leer usando **apuntador X**
- LD Rd, Y: Leer usando **apuntador Y**
- LD Rd, Z: Leer usando **apuntador Z**
- LD Rd, X+: Leer y **post-incrementar X**
- LD Rd, -X: **Pre-decrementar X** y leer
- LD Rd, Y+q: Leer con **desplazamiento q**



Instrucción LD - Load Indirect

LD Rd, X/Y/Z

Carga un **byte** usando **direccionamiento indirecto** con **apuntadores**

Ejemplos:

```
1 ; Configurar apuntador X
2 LDI r26, low(0x0100) ; Parte baja de la direccion
3 LDI r27, high(0x0100) ; Parte alta de la direccion
4
5 LD r16, X ; Leer desde dir. apuntada por X
6 LD r17, X+ ; Leer y post-incrementar X
7 LD r18, -Y ; Pre-decrementar Y y leer
8 LD r19, Z+2 ; Leer con offset +2
```


Instrucción IN - Read from I/O

IN Rd, P

Lee un **byte** desde un **puerto de E/S** a un **registro**

Características:

- Accede a **registros de E/S** (0x00-0x3F)
- Tiempo de ejecución: 1 ciclo de reloj
- **Más rápido** que **LDS** para registros de E/S

Ejemplos comunes:

```
1 IN r16, PINB      ; Leer estado del Puerto B
2 IN r17, PINC      ; Leer estado del Puerto C
3 IN r18, PIND      ; Leer estado del Puerto D
4 IN r19, SREG      ; Leer registro de estado
5 IN r20, SPL       ; Leer stack pointer bajo
6 IN r21, SPH       ; Leer stack pointer alto
```

Instrucción LPM - Load Program Memory

LPM / LPM Rd, Z / LPM Rd, Z+

Carga un **byte** desde la **memoria flash** (programa) usando el **apuntador Z**

Variantes:

- LPM: Carga en **R0**
- LPM Rd, Z: Carga en **registro Rd**
- LPM Rd, Z+: Carga en **Rd** y **post-incrementa Z**

Ejemplo - Leer tabla de constantes:

```

1 ; Definir tabla en memoria Flash
2 .cseg
3 tabla: .db 10, 20, 30, 40, 50
4
5 LDI ZL, low(tabla*2)      ; Cargar direccion (x2 por bytes)
6 LDI ZH, high(tabla*2)
7 LPM r16, Z                ; Leer primer elemento (10)
8 LPM r17, Z+              ; Leer segundo elemento (20)

```

Instrucción STS - Store Direct to SRAM

STS k , Rr

Almacena el contenido de un registro en la **SRAM**

Características:

- Acceso directo a **cualquier posición** de **SRAM**
- Dirección k puede ser de 0x0000 a 0xFFFF
- Tiempo de ejecución: 2 ciclos de reloj



Instrucción STS - Store Direct to SRAM

STS k, Rr

Almacena el contenido de un registro en la **SRAM**

Ejemplos:

```
1 LDI r16, 0xAA           ; Cargar valor
2 STS 0x0100, r16         ; Guardar en direccion 0x0100
3
4 LDI r17, 0x55
5 STS variable, r17       ; Guardar en variable
6
7 ; Intercambio de datos
8 LDS r18, var1           ; Leer variable 1
9 LDS r19, var2           ; Leer variable 2
10 STS var1, r19          ; Escribir var2 en var1
11 STS var2, r18          ; Escribir var1 en var2
```

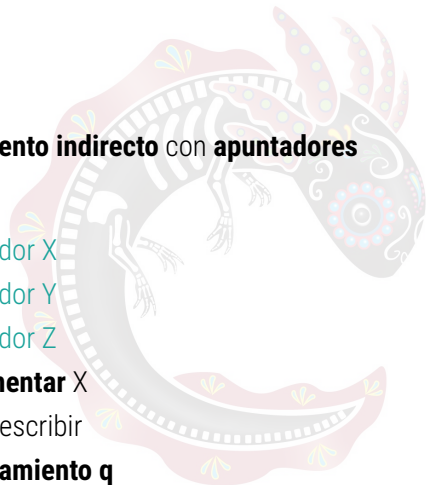
Instrucción ST - Store Indirect

ST X/Y/Z, Rr

Almacena un **byte** usando **direccionamiento indirecto** con **apuntadores**

Variantes disponibles:

- ST X, Rr: Escribir usando **apuntador X**
- ST Y, Rr: Escribir usando **apuntador Y**
- ST Z, Rr: Escribir usando **apuntador Z**
- ST X+, Rr: Escribir y **post-incrementar X**
- ST -X, Rr: **Pre-decrementar X** y escribir
- ST Y+q, Rr: Escribir con **desplazamiento q**



Instrucción ST - Store Indirect

ST X/Y/Z, Rr

Almacena un **byte** usando **direccionamiento indirecto** con **apuntadores**

Ejemplo - Llenar buffer:

```
1 ; Configurar puntero Y para buffer
2 LDI YL, low(buffer)
3 LDI YH, high(buffer)
4 LDI r16, 0x00 ; Valor inicial
5
6 ; Llenar 10 posiciones
7 llenar:
8 ST Y+, r16 ; Escribir y avanzar puntero
9 INC r16 ; Incrementar valor
10 CPI YL, low(buffer+10) ; Llegamos al final?
11 BRNE llenar ; No, continuar
```

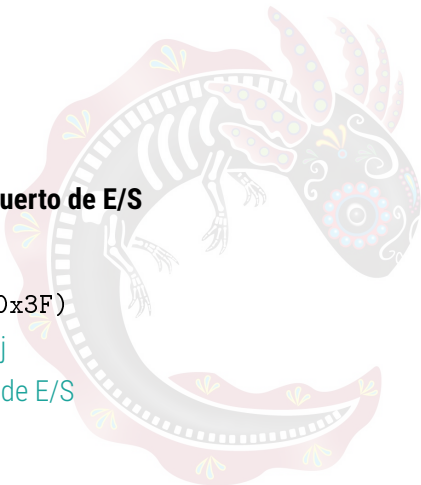
Instrucción OUT - Write to I/O

OUT P, Rr

Escribe un **byte** desde un **registro** a un **puerto de E/S**

Características:

- Accede a **registros de E/S** (0x00-0x3F)
- Tiempo de ejecución: 1 ciclo de reloj
- **Más rápido** que **STS** para registros de E/S



Instrucción OUT - Write to I/O

OUT P, Rr

Escribe un **byte** desde un **registro** a un **puerto de E/S**

Ejemplos - Control de puertos:

```
1 ; Configurar Puerto B como salida
2 LDI r16, 0xFF
3 OUT DDRB, r16 ; Todos los pines como salida
4
5 ; Encender LEDs en Puerto B
6 LDI r17, 0b10101010
7 OUT PORTB, r17 ; Patron alternado
8
9 ; Configurar Puerto C (pines 0-3 entrada, 4-5 salida)
10 LDI r18, 0b00110000
11 OUT DDRC, r18
```


Instrucciones PUSH y POP

Stack Operations

Manejan el **stack** (pila) para **almacenamiento temporal** de registros

Operaciones:

- PUSH Rr: **Guarda registro** en el **stack**
- POP Rd: **Recupera registro** del **stack**
- El stack **crece hacia abajo** en **memoria**
- Útil en **interrupciones** y **subrutinas**



Instrucciones PUSH y POP

Stack Operations

Manejan el **stack** (pila) para **almacenamiento temporal** de registros

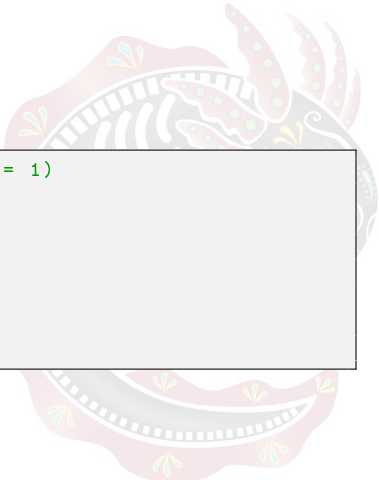
Ejemplo - Subrutina que preserva registros:

```
1  mi_funcion:
2      PUSH  r16                ; Guardar registros
3      PUSH  r17
4      PUSH  r18
5
6      ; ... codigo de la funcion ...
7
8      POP   r18                ; Restaurar en orden inverso
9      POP   r17
10     POP   r16
11     RET                      ; Retorno
```

Ejemplo 1: Lectura de Pulsadores

```
1 ; Configuracion inicial
2 .def temp = r16
3 .def buttons = r17
4
5 main:
6 ; Configurar Puerto D como entrada (pulsadores)
7     LDI temp, 0x00
8     OUT DDRD, temp          ; Todo como entrada
9
10 ; Activar pull-ups internos
11     LDI temp, 0xFF
12     OUT PORTD, temp         ; Activar pull-ups
13
14 ; Configurar Puerto B como salida (LEDs)
15     LDI temp, 0xFF
16     OUT DDRB, temp          ; Todo como salida
17 read_loop:
18 ; Leer estado de pulsadores
19     IN buttons, PIND         ; Leer Puerto D
```

Ejemplo 1: Lectura de Pulsadores



```
1 ; Invertir bits (pulsador presionado = 1)
2   COM buttons
3
4 ; Mostrar en LEDs (Puerto B)
5   OUT PORTB, buttons
6
7 ; Repetir
8   RJMP read_loop
```

Ejemplo 2: Contador en Display 7 Segmentos

```
1 ; Tabla de patrones para display 7 segmentos
2 .cseg
3 .org 0x0000
4
5 ; Tabla en memoria Flash (0-9)
6 tabla_7seg: .db 0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07
   , 0x7F, 0x6F
7
8 main:
9 ; Configurar Puerto B como salida
10     LDI r16, 0xFF
11     OUT DDRB, r16
12
13 ; Inicializar contador
14     LDI r17, 0           ; Contador 0-9
15
16 contador_loop:
17 ; Calcular direccion en tabla
18     LDI ZL, low(tabla_7seg*2)
19     LDI ZH, high(tabla_7seg*2)
20     ADD ZL, r17           ; Sumar offset
```

Ejemplo 2: Contador en Display 7 Segmentos

```
1 ; Leer patron de la tabla
2   LPM r18, Z
3
4 ; Mostrar en display
5   OUT PORTB, r18
6
7 ; Delay (simplificado)
8   CALL delay_1s
9
10 ; Incrementar contador
11   INC r17
12   CPI r17, 10           ; Llego a 10?
13   BRNE contador_loop
14   CLR r17               ; Reiniciar a 0
15   RJMP contador_loop
```

Comparación de Rendimiento

Instrucción	Ciclos	Palabras	Uso Recomendado
LDI	1	1	Cargar constantes
MOV	1	1	Copiar entre registros
IN/OUT	1	1	Registros de E/S (0x00-0x3F)
LDS/STS	2	2	SRAM y E/S extendida
LD/ST	1-2	1	Direccionamiento indirecto
LPM	3	1	Leer constantes de Flash

Recomendación

Usar **IN/OUT** para **registros de E/S** cuando sea posible, son más rápidos que LDS/STS

Mejores Prácticas

1. Uso eficiente de registros:

```
1 ; Mal - desperdiciar registros
2 LDI r16, 0xFF
3 STS DDRB, r16
4 LDI r17, 0xFF ; Redundante
5
6 ; Bien - reutilizar registros
7 LDI r16, 0xFF
8 OUT DDRB, r16 ; Mas rapido que STS
9 OUT DDRC, r16 ; Reutilizar r16
```


Mejores Prácticas

2. Aprovechar post/pre incremento:

```
1 ; Mal - incremento manual
2 LD r16, X
3 LDI r17, 1
4 ADD r26, r17
5 ADC r27, ZH ; Manejar carry
6
7 ; Bien - post-incremento automatico
8 LD r16, X+ ; Mas eficiente
```

Gestión de Memoria

Organización típica de memoria:

- **0x0100-0x01FF:** Variables globales
- **0x0200-0x02FF:** Buffers de datos
- **0x0300-0x08FF:** Stack (crece hacia abajo)

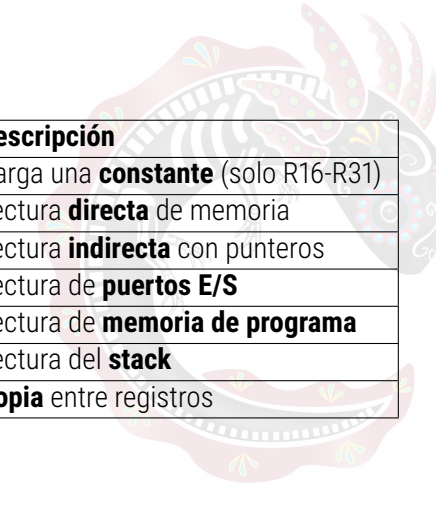
Inicialización del stack pointer:

```
1 ; Configurar SP al final de SRAM
2 LDI r16, low(RAMEND)
3 OUT SPL, r16
4 LDI r16, high(RAMEND)
5 OUT SPH, r16
```

Importante

Siempre inicializar el **stack pointer** antes de usar **CALL, RET, PUSH** o **POP**

Resumen de Instrucciones de Lectura



Instrucción	Fuente	Descripción
LDI Rd, k	Inmediato	Carga una constante (solo R16-R31)
LDS Rd, k	SRAM	Lectura directa de memoria
LD Rd, X/Y/Z	SRAM	Lectura indirecta con punteros
IN Rd, P	E/S	Lectura de puertos E/S
LPM Rd, Z	Flash	Lectura de memoria de programa
POP Rd	Stack	Lectura del stack
MOV Rd, Rr	Registro	Copia entre registros

Resumen de Instrucciones de Escritura

Instrucción	Destino	Descripción
STS k, Rr	SRAM	Escritura directa a memoria
ST X/Y/Z, Rr	SRAM	Escritura indirecta con punteros
OUT P, Rr	E/S	Escritura a puertos E/S
PUSH Rr	Stack	Escritura al stack
SPM	Flash	Escritura a memoria de programa

Puntos clave:

- Usar **IN/OUT** para **registros de E/S** (más rápido)
- **LDI** solo funciona con **R16-R31**
- Los **apuntadores X, Y, Z** permiten **acceso flexible** a memoria
- El **stack** es **esencial** para interrupciones y subrutinas